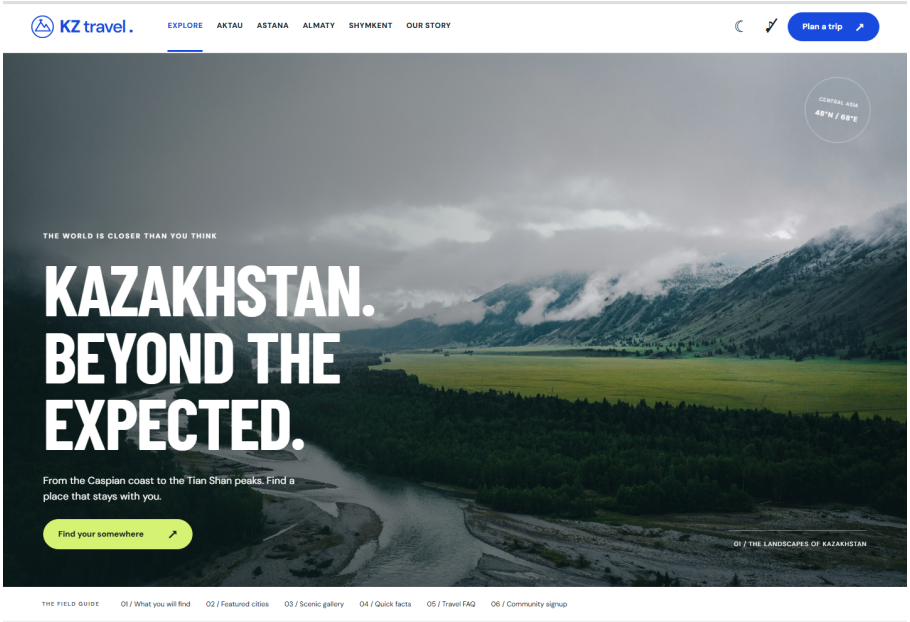
Find your next somewhere.

Assignment 6: jQuery



Published KZ Travel homepage, captured after the Assignment 6 deployment.

Team	KZ Travel
Members	Ramazan Utegen Ziyadinkhan Kudaibergenuly Ruzimuhammad
Group	SE-2501

Objective. Continue the team's Kazakhstan travel project and use jQuery to simplify DOM manipulation, event handling and animations. Implement Tasks 0-9 and publish the updated six-page website.

Deployed website: https://ramazanutegen.github.io/assignment1web/

Repository: github.com/RamazanUtegen/assignment1web

A more useful travel guide.

The same team and original project topic are retained. Six existing pages, original photographs, city guides, tables, navigation, forms, clock, saved places, themes, ratings, route planner and galleries remain available. The jQuery features fit the existing travel experience.

Task	Requirement	Location
0	CDN + document.ready	All pages; browser console
1	Real-time search and live filter	Home destination cards
2	Autocomplete suggestions	Destination search combobox
3	Search highlighting	Destination text and FAQ answers
4	Stylized scroll progress bar	Top edge of all pages
5	Animated number counters	Home guide statistics
6	Loading spinner on submit	Signup, city and contact forms
7	Temporary fading notifications	Save, form and copy actions
8	Copy to clipboard + tooltip	Guide links and generated route
9	Lazy image loading	Below-fold photos and galleries
Part 4	Commit, push and deployment	Existing GitHub Pages website

Run locally. Open index.html or run python -m http.server 8765 --bind 127.0.0.1 in the project folder. No build or backend is needed. jQuery uses the specified CDN and an identical local fallback. Forms simulate processing and send no personal information.

Main files. script.js implements the jQuery features; css/assignment6.css supplies their styles. The earlier validator and saved-place controls communicate with the new module through custom events.

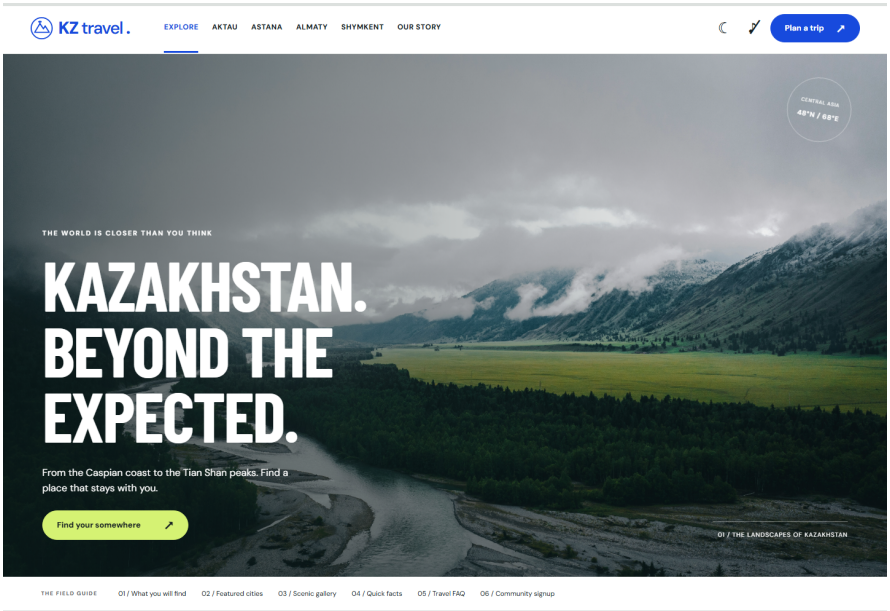
KZ TRAVEL / ASSIGNMENT 6 / TASK 0

Set up jQuery.

1. Include the specified jQuery 4.0.0 CDN script inside each HTML head, followed by external script.js. defer preserves execution order; an integrity hash verifies the CDN bytes.

2. In script.js, call $(document).ready() and log jQuery is ready! before attaching the handlers.

3. If the CDN cannot load, initialise with the bundled identical jQuery version. Browser checks verify both CDN and blocked-CDN paths.



Webpage result. The homepage runs with jQuery 4.0.0. The browser console log is verified in the saved test evidence.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 0. CDN setup and DOM readiness

index.html

```
33   <!-- Assignment 6, Task 0: CDN first, local fallback in external script.js. -->
34   <script defer src="https://code.jquery.com/jquery-4.0.0.min.js" integrity="sha256-
   OaVG6prZf4v69dPg6PhVattBXkcOWQB62pdZ3ORyrao=" crossorigin="anonymous"></script>
35   <script defer src="script.js?v=6"></script>
```

script.js

```
5     function initialise() {
6         const $ = window.jQuery;
7         // Task 0: wait for the DOM before binding any jQuery handlers.
8         $(document).ready(function () {
9             console.log('jQuery is ready!');
10            const motion = window.matchMedia('(prefers-reduced-motion: reduce)');
```

Code screenshot from the actual project, with original file names and line numbers.

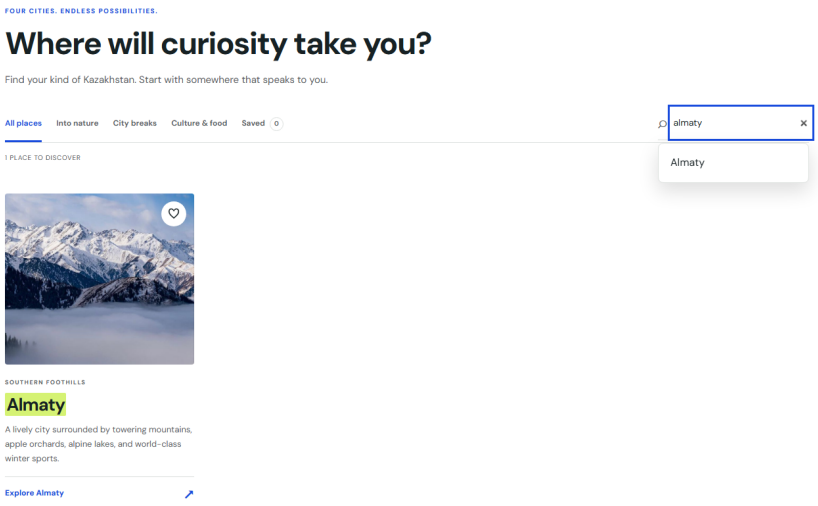
KZ TRAVEL / ASSIGNMENT 6 / TASK 1

Search as you type.

1. Bind the destination search input with jQuery .on(input). Normalize the query and read searchable city names and tags from the cards.

2. Use .filter() to combine the text with the selected category or saved-state filter, then update hidden properties, the results count and the empty state.

3. Test Almaty, combine a mismatching category, and clear the filters. The cards update immediately without reloading.



Webpage result. Only Almaty remains; its matching name is highlighted.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 1. Live search with combined category filters

script.js

```
74              function renderDestinations(showSuggestions = false) {
75                  const query = queryText($search);
76                  const $matches = $cards.filter(function () {
77                      return inCategory(this) && $(this).attr('data-search-text').includes(query);
78                  });
79                  $cards.each(function () {
80                      $(this).closest('.destination-column').prop('hidden', !$matches.is(this));
81                  });
82                  $('[data-results-count]').text(`${$matches.length} ${$matches.length === 1 ? 'place' :
    'places'} to discover`);
83                  $('[data-no-results]').prop('hidden', $matches.length !== 0);
84                  $filters.each(function () {
85                      const active = $(this).attr('data-filter') === category;
86                      $(this).toggleClass('is-active', active).attr('aria-pressed', String(active));
87                  });
88                  highlight($cards.find('.card-title, .card-text'), query);
89                  $suggestions.empty();
90                  closeSuggestions();
91                  if (showSuggestions && query && $matches.length) {
92                      $matches.each(function (index) {
93                          $('<li>', { role: 'option', id: `destination-option-${index}`,
94                              'aria-selected': 'false', text: $(this).find('.card-title').text().trim()
95                          }).appendTo($suggestions);
```

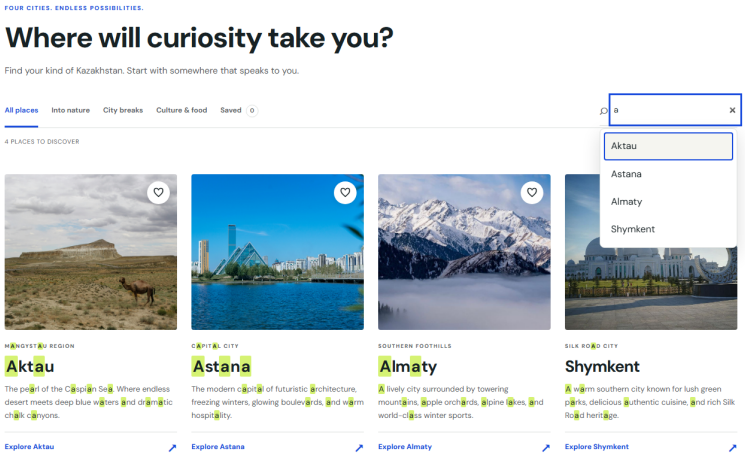
Code screenshot from the actual project, with original file names and line numbers.

Offer the next useful word.

1. Build a listbox of city names from the same matching set, using .text() to create safe labels. Show it only for a non-empty query.

2. Use Arrow Down/Up and aria-activedescendant to select an option. Enter or a click applies the suggestion and filters the cards.

3. Close the dropdown on Escape or blur. Test keyboard selection, empty results and clearing the input.



Webpage result. Typing a shows matching suggestions with the first keyboard option selected.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 2. Autocomplete: safe suggestions and keyboard selection

script.js

```
91                if (showSuggestions && query && $matches.length) {
92                    $matches.each(function (index) {
93                        $('<li>', { role: 'option', id: `destination-option-${index}`,
94                            'aria-selected': 'false', text: $(this).find('.card-title').text().trim()
95                        }).appendTo($suggestions);
96                    });
97                    $suggestions.prop('hidden', false);
98                    $search.attr('aria-expanded', 'true');
99                }
100               scheduleViewport();
```

script.js

```
125               if (event.key === 'Enter' && activeSuggestion >= 0) {
126                   event.preventDefault();
127                   chooseSuggestion($suggestions.children().eq(activeSuggestion));
128                   return;
129               }
130               if (!['ArrowDown', 'ArrowUp'].includes(event.key)) return;
131               if ($suggestions.prop('hidden')) renderDestinations(true);
132               const $options = $suggestions.children();
133               if (!$options.length) return;
134               event.preventDefault();
135               activeSuggestion = activeSuggestion < 0 ? (event.key === 'ArrowDown' ? 0 : $options.length
    - 1) :
136                   (activeSuggestion + (event.key === 'ArrowDown' ? 1 : -1) + $options.length) %
    $options.length;
137               $options.attr('aria-selected', 'false').eq(activeSuggestion).attr('aria-selected', 'true');
138               $search.attr('aria-activedescendant', $options.eq(activeSuggestion).attr('id'));
139           });
```

Code screenshot from the actual project, with original file names and line numbers.

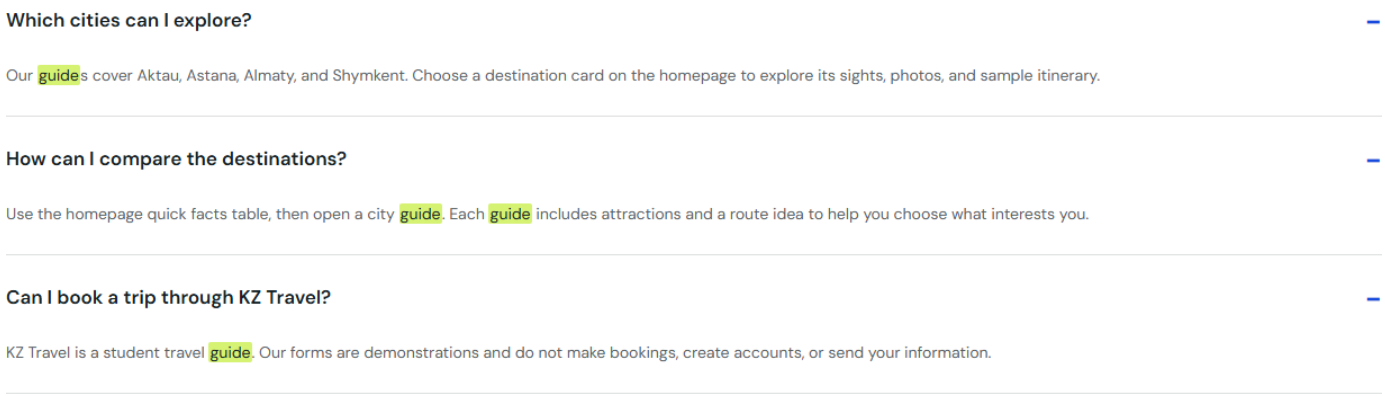
KZ TRAVEL / ASSIGNMENT 6 / TASK 3

Make every match visible.

1. Add a FAQ search that checks questions and full answers, including collapsed text. Reveal matching answers and report the matching-question count.

2. Escape regex metacharacters and traverse text nodes. Replace every case-insensitive match with a mark element; user input is never interpreted as HTML.

3. Remove previous marks before each update. On clear, restore all FAQ items and their previous open/closed state. Test guide, [and HTML-like input.



Webpage result. Every occurrence of guide in matching questions and answers is marked.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 3. Highlight literal matches without injecting HTML

script.js

```
35                const pattern = new RegExp(query.replace(/[.*+?^${}()|[\]\\]/g, '\\$&'), 'gi');
36                $containers.each(function () {
37                    const walker = document.createTreeWalker(this, NodeFilter.SHOW_TEXT);
38                    const nodes = [];
39                    while (walker.nextNode()) nodes.push(walker.currentNode);
40                    nodes.forEach(node => {
41                        pattern.lastIndex = 0;
42                        const matches = [...node.textContent.matchAll(pattern)];
43                        if (!matches.length) return;
44                        const fragment = document.createDocumentFragment();
45                        let offset = 0;
46                        matches.forEach(match => {
47                            fragment.append(node.textContent.slice(offset, match.index));
48                            fragment.append($('<mark>', { class: 'search-match', text: match[0] })[0]);
49                            offset = match.index + match[0].length;
50                        });
51                        fragment.append(node.textContent.slice(offset));
52                        node.replaceWith(fragment);
53                    });
54                });
55            }
56
57            const $search = $('[data-destination-search]');
```

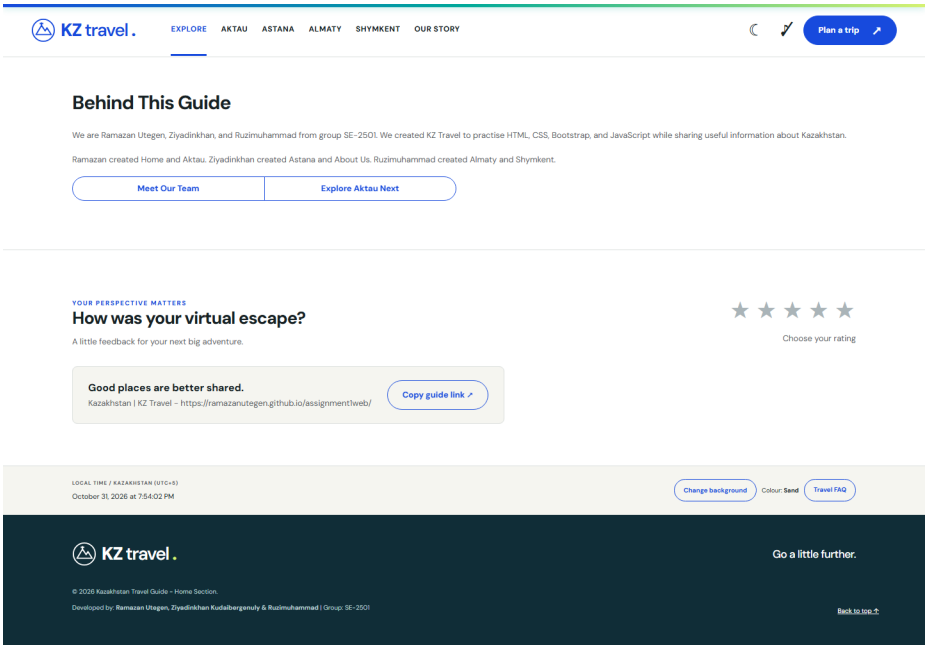
Code screenshot from the actual project, with original file names and line numbers.

Show reading progress.

1. Add a fixed horizontal bar with a blue, teal and lime gradient on every page. Expose its 0-100 value through an accessible progressbar role.

2. On jQuery scroll/resize events, schedule one update per animation frame. Calculate scrollTop / (scrollHeight - innerHeight) * 100.

3. Clamp the percentage, handle pages without scrollable distance and set the span width with .css(). Verify that the bottom of the page reads 100%.



Webpage result. The colored bar reaches the full viewport width at the page bottom.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 4. Scroll progress: clamped percentage and styled width

script.js

```
238             function updateViewport() {
239                 viewportFrame = 0;
240                 const distance = document.documentElement.scrollHeight - window.innerHeight;
241                 const percent = distance > 0 ? Math.max(0, Math.min(100, $(window).scrollTop() / distance *
    100)) : 100;
242                 $progress.attr('aria-valuenow', Math.round(percent)).find('span').css('width', percent +
    '%');
243                 $counters.filter(function () { return !$(this).attr('data-counted') && nearViewport(this);
    }).each(function () {
```

script.js

```
255             function scheduleViewport() {
256                 if (!viewportFrame) viewportFrame = requestAnimationFrame(updateViewport);
257             }
258             $(window).on('scroll resize load', scheduleViewport);
259             // Load the next slide before Bootstrap's transition begins.
```

Code screenshot from the actual project, with original file names and line numbers.

Bring the guide numbers to life.

1. Display three real project counts: 4 city destinations, 6 pages and 9 photos in the homepage carousel.

2. When the block enters the viewport, use jQuery .animate() on a numeric object. The step callback updates the displayed integer and complete sets the exact target.

3. Mark each counter as counted to avoid repeats. Reduced-motion preference skips the movement and shows the final values immediately.



Webpage result. Counters finish at 4, 6 and 9; static accessible labels keep the counts understandable.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 5. Animate each counter once when it enters view

script.js

```
243                 $counters.filter(function () { return !$(this).attr('data-counted') && nearViewport(this);
    }).each(function () {
244                     const $counter = $(this).attr('data-counted', 'true');
245                     const target = Number($counter.attr('data-counter'));
246                     $({ value: 0 }).animate({ value: target }, {
247                         duration: duration(1400), easing: 'swing',
248                         step(value) { $counter.text(Math.floor(value)); },
249                         complete() { $counter.text(target); }
250                     });
251                 });
252                 $pendingImages.filter(function () { return nearViewport(this, 120); }).each(function () {
    loadImage(this); });
253                 $pendingImages = $pendingImages.filter('[data-src]');
```

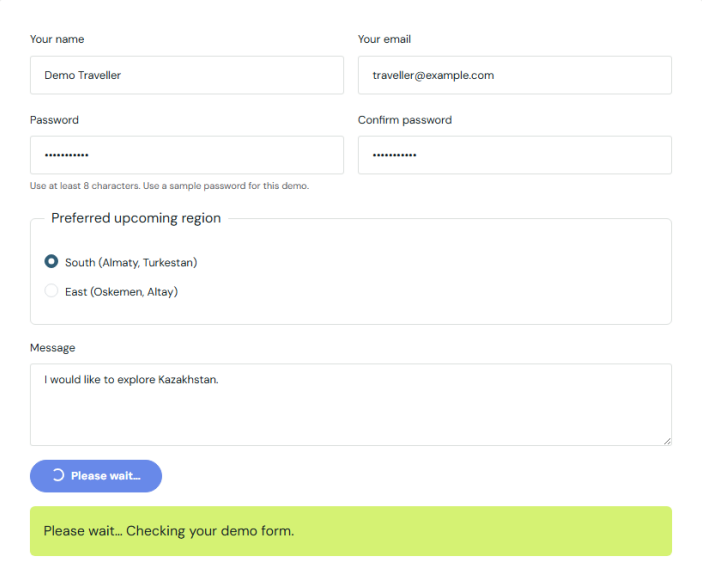
Code screenshot from the actual project, with original file names and line numbers.

Make form processing clear.

1. Keep the existing validation. Invalid fields show errors immediately; successful validation dispatches a cancelable form-loading event to the jQuery handler.

2. Replace the submit button contents with a spinner and Please wait, disable the button and mark the form aria-busy. Repeated submit events are ignored.

3. After the 1.4-second simulated delay, restore the original button and run the completion callback. Clear sample passwords and show the demo confirmation and toast.



Webpage result. The valid signup form is busy and the submit control is disabled.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 6. Validated form: spinner, disabled button and completion

script.js

```
167             $(document).on('kz:form-loading', '[data-demo-form]', function (event) {
168                 event.preventDefault();
169                 const $form = $(this);
170                 if ($form.attr('aria-busy') === 'true') return;
171                 const $button = $form.find('button[type=submit]');
172                 const original = $button.contents().detach();
173                 $form.attr('aria-busy', 'true');
174                 $form.find('.form-status').text('Please wait… Checking your demo form.');
175                 $button.prop('disabled', true).empty()
176                     .append($('<span>', { class: 'submit-spinner', 'aria-hidden': 'true' }))
177                     .append(document.createTextNode('Please wait…'));
178                 setTimeout(() => {
179                     $button.empty().append(original).prop('disabled', false);
180                     $form.attr('aria-busy', 'false');
181                     event.originalEvent.detail.complete();
182                     notify('Form checked successfully. Demo only; no information was sent.');
183                 }, 1400);
184             });
185
186             // Task 8: copy actual guide/route text; acknowledge only real success.
187             $('[data-copy]').on('click', async function () {
```

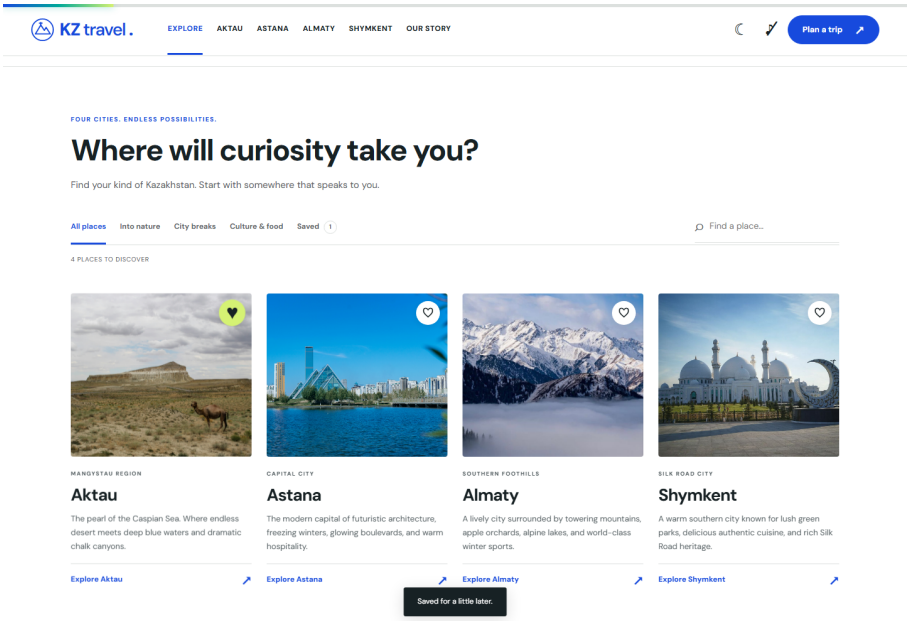
Code screenshot from the actual project, with original file names and line numbers.

Confirm the action, then fade.

1. Share one accessible status toast between saved places, successful form checks and copy actions. Earlier controls dispatch a notification event.

2. Set the message with .text() and show it using .fadeIn(). Cancel the previous timer and animation queue before displaying a newer message.

3. After 2.8 seconds, use .fadeOut() and clear the text. Browser checks verify that saving a city displays a message and the toast later disappears.



Webpage result. Saving Aktau produces a temporary confirmation at the bottom of the viewport.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 7. Shared toast with jQuery fade and timer cancellation

script.js

```
14  const $toast = $('.toast-message');
15  let toastTimer;
16  function notify(message) {
17      clearTimeout(toastTimer);
18      $toast.stop(true, true).text(message).fadeIn(duration(160));
19      toastTimer = setTimeout(() => {
20          $toast.fadeOut(duration(350), function () { $(this).empty(); });
21      }, 2800);
22  }
23  $(document).on('kz:notify', function (event) {
24      notify(event.originalEvent.detail.message);
25  });
26
```

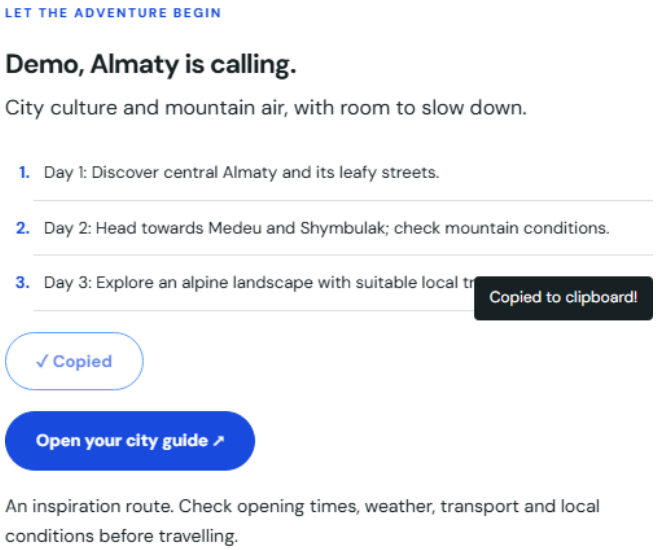
Code screenshot from the actual project, with original file names and line numbers.

Copy something worth sharing.

1. Add Copy guide link on every page and Copy my route inside the generated route result. Read the actual displayed text from the linked target element.

2. Use navigator.clipboard.writeText() in a secure context, with a legacy text-selection fallback. Await successful copying before showing a checkmark and Copied to clipboard tooltip.

3. Restore the button after 2.2 seconds. Test actual clipboard text and denied permission; failures show a clear message without claiming success.



Webpage result. A generated three-day Almaty route is copied successfully.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 8. Clipboard API, fallback and truthful confirmation

script.js

```
195                     const text = target.innerText.trim();
196                     if (navigator.clipboard && window.isSecureContext) {
197                         await navigator.clipboard.writeText(text);
198                     } else {
199                         const focused = document.activeElement;
200                         const $buffer = $('<textarea>', { text, class: 'visually-hidden', readonly: true
    }).appendTo('body');
201                         $buffer[0].select();
202                         const copied = document.execCommand('copy');
203                         $buffer.remove();
204                         focused?.focus();
205                         if (!copied) throw new Error('Clipboard unavailable');
206                     }
207                     $button.text('✓ Copied');
208                     $tooltip.text('Copied to clipboard!').prop('hidden', false);
209                     notify('Copied to clipboard! Your next adventure is ready to share.');
210                 } catch {
211                     $tooltip.text('Copy unavailable. Select and copy the text.').prop('hidden', false);
212                     notify('Clipboard access was unavailable. You can select and copy the text.');
213                 } finally {
214                     setTimeout(() => {
215                         $button.text(original).prop('disabled', false);
```

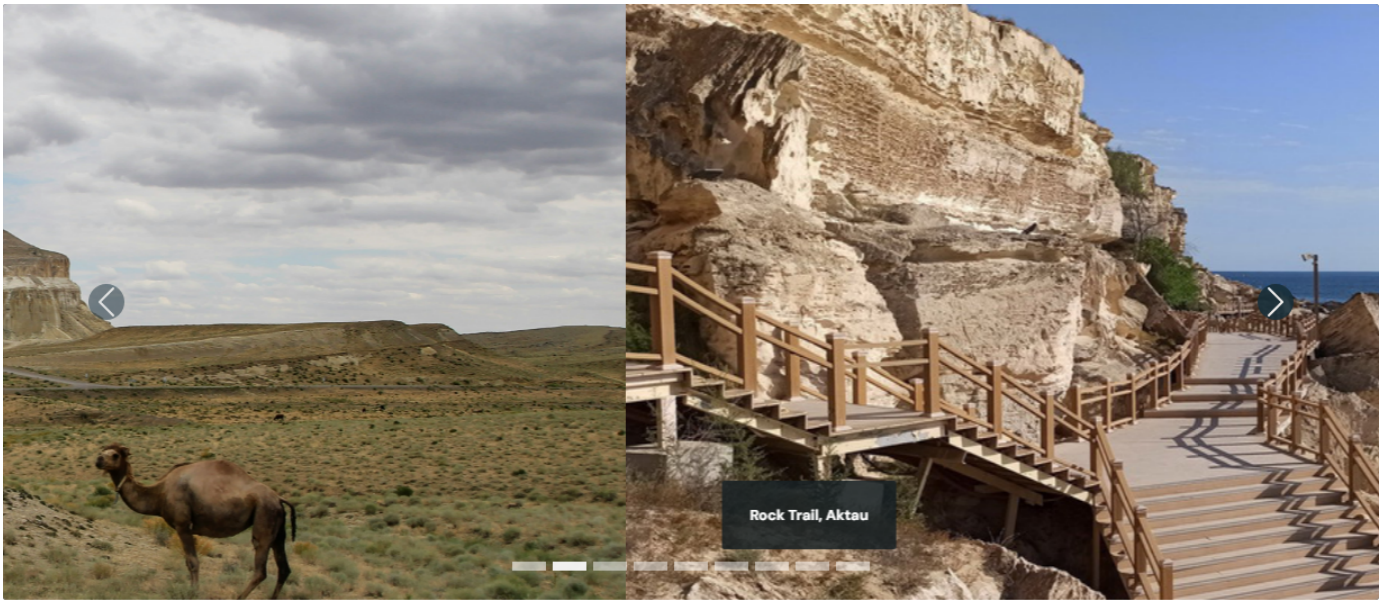
Code screenshot from the actual project, with original file names and line numbers.

Load the view as it arrives.

1. Preserve each original photograph, alt text and dimensions. Store its URL in data-src and initially use a small local placeholder; retain noscript image copies.

2. Use jQuery viewport checks on throttled scroll/resize events. For an image near the visible region, set src from data-src with .attr() and remove data-src.

3. Preload an upcoming Bootstrap slide before its transition. Network checks verify that Rock Trail is absent from initial requests and loads after the gallery is visited.



Webpage result. The next carousel photograph has loaded while offscreen images remain deferred.

KZ TRAVEL / ASSIGNMENT 6 / ACTUAL SOURCE

Task 9. Viewport lazy loading with jQuery attributes

script.js

```
226             function nearViewport(element, margin = 0) {
227                 if (!$(element).is(':visible')) return false;
228                 const bounds = element.getBoundingClientRect();
229                 return bounds.top < window.innerHeight + margin && bounds.bottom > -margin;
230             }
231             function loadImage(image) {
232                 const $image = $(image);
233                 if (!$image.attr('data-src')) return;
234                 $image.one('load', function () { $(this).attr('data-lazy-state', 'loaded');
    scheduleViewport(); });
235                 $image.one('error', function () { $(this).attr('data-lazy-state', 'error'); });
236                 $image.attr('data-lazy-state', 'loading').attr('src', $image.attr('data-
    src')).removeAttr('data-src');
237             }
238             function updateViewport() {
239                 viewportFrame = 0;
```

script.js

```
252                 $pendingImages.filter(function () { return nearViewport(this, 120); }).each(function () {
    loadImage(this); });
253                 $pendingImages = $pendingImages.filter('[data-src]');
```

Code screenshot from the actual project, with original file names and line numbers.

Published and checked.

The website implementation was committed and pushed to the existing shared main branch. GitHub Pages rebuilt the site successfully. The report was prepared after checking the published version.

Verified website source: 7c49e3e437efffaeb09b2275005b7d43de051b8f

Verified: 2026-10-31 19:53:58 +05:00 (Asia/Tashkent).

Live URL: https://ramazanutegen.github.io/assignment1web/

Pages workflow: Open successful build and deployment

HTTP verification. 60 published HTML/assets returned HTTP 200 and matched the checkout. Exact hashes and the workflow result are stored in docs/assignment-6/deployment.json.

Browser verification. The local and published websites were checked in Chromium and Microsoft Edge, each at 1440 x 1000 and 390 x 844. All six pages fit the viewport and retained team credits; the test recorded no JavaScript exceptions or HTTP errors.

- Search, category/Saved combinations, no-results recovery, keyboard suggestions and literal special-character/HTML-like queries.

- FAQ highlights, restored accordion state, night mode, counters and reduced-motion values.

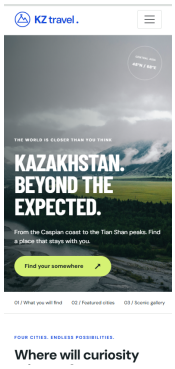
- Invalid and valid forms, a disabled submit button, repeated-submit rejection and completion recovery.

- Fading toast, actual Clipboard API contents in Chromium, route and guide copying, reset state and clipboard denial.

- Deferred image network requests, successful carousel lazy loading, scroll progress to 100%, responsive layout and contact dialog Escape.

- Blocked CDN with working local jQuery fallback.

Evidence files. docs/assignment-6/browser-verification.json and live-browser-verification.json contain repeatable scenario results. Source and webpage screenshots accompany this report in the ZIP. The test script is scripts/verify-assignment6.cjs.



Mobile homepage, Chromium at 390 x 844.

What we learned.

Integrating an existing project. Search and notifications already had JavaScript handlers. Adding competing handlers would cause inconsistent results. We gave jQuery ownership of the discovery display and connected saved-place changes through events. One shared toast now handles confirmation messages.

Safe highlighting. A raw search value can break a regular expression, and inserting it as HTML can create unintended elements. Escaping regex syntax and constructing marks from text nodes supports literal queries while preserving the document structure.

State and asynchronous feedback. FAQ searching temporarily expands matching answers, so we save the original accordion state and restore it on clear. Forms validate first, then expose a busy state and use a completion callback. Clipboard confirmation follows the resolved promise and reports permission failure honestly.

Media and performance. Hidden carousel slides are not in the viewport. We load their images before the slide transition in addition to normal scroll checks. A single animation-frame scheduler groups progress, counter and image work. Reduced-motion preference changes animation duration to zero.

Defence sequence. Open the console for Task 0; filter Almaty; choose a suggestion by keyboard; search guide in the FAQ; scroll through the progress bar and counters; submit invalid then valid sample data; save a city; copy a generated route; inspect deferred requests in Network. Explain .on(), .filter(), .text(), .attr(), .prop(), .animate() and .fadeOut().

Delivery. Submit this PDF and KZ-Travel-Assignment-6.zip. Every member uploads the same complete files individually and prepares to explain every feature. Check the exact deadline in the LMS; the supplied assignment PDF does not specify a date.

Limitations. This is a static educational travel guide. Processing is simulated; no account, booking, subscription or message is created. Actual clipboard access follows browser permissions. Dynamic travel details should be checked before a real trip.

References. The supplied Assignment 6 jQuery.pdf, Tasks 0-9 and Part 4; official jQuery documentation:

jQuery .ready() - https://api.jquery.com/ready/

jQuery .filter() - https://api.jquery.com/filter/

jQuery .on() - https://api.jquery.com/on/

jQuery .animate() - https://api.jquery.com/animate/

jQuery .fadeOut() - https://api.jquery.com/fadeOut/

jQuery is MIT licensed (vendor/jquery-LICENSE.txt). Bootstrap and local font licences remain in the project. All original project images are preserved. Russian defence notes are included in docs/assignment-6-defense-ru.md.